

Scaling Laws for a Character-Level GPT Model

Priyanshu Kumar

Abstract

This project implements a character-level GPT model using PyTorch and investigates neural scaling laws by training models of different parameter sizes. Validation loss is analyzed as model size increases, and empirical scaling exponents are estimated.

1 Embedding

Each input token is mapped to a dense embedding vector of dimension 64, 128, or 256, depending on the model configuration.

2 Positional Encoding

Since the self-attention mechanism has no inherent notion of word order, positional information is added to each token embedding using sinusoidal positional encoding.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

where

- pos is the token position.
- i is the embedding dimension index.
- d_{model} is the embedding dimension (64, 128 or 256).

3 Masked Self-Attention

Each embedding vector is projected into Query (Q), Key (K), and Value (V) vectors using learnable weight matrices.

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

The attention score is computed by taking the dot product of the Query vector with the transpose of the Key vector and scaling it by $\sqrt{d_k}$.

$$d_k = \frac{d_{\text{model}}}{\text{num_heads}}$$

The attention computation follows these steps:

1. Compute the scaled dot-product attention scores.
2. Add the causal mask to prevent attention to future tokens.
3. Apply the softmax function to obtain attention weights.
4. Multiply the attention weights by the Value matrix to obtain contextual representations.

The masked attention equation is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

4 Multi-Head Attention

Instead of computing a single attention function, multiple attention heads are computed in parallel.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

The outputs of all heads are concatenated and projected using a learnable output projection matrix.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

5 Feed Forward Network

Each contextual representation is independently processed through a position-wise feed-forward neural network.

$$\text{FFN}(X) = \text{GELU}(XW_1 + b_1)W_2 + b_2$$

where

$$W_1 \in R^{d_{\text{model}} \times d_{\text{ff}}}, \quad W_2 \in R^{d_{\text{ff}} \times d_{\text{model}}},$$

and

$$d_{\text{ff}} = 4d_{\text{model}}.$$

6 Pre-Layer Normalization

To improve training stability and gradient flow, Layer Normalization is applied before each sub-layer.

$$x^{(k+1)} = x^{(k)} + \text{SubLayer} \left(\text{LN}(x^{(k)}) \right)$$

7 Dataset

The experiments were conducted using the Tiny Shakespeare dataset. The dataset consists of approximately 90,000 training samples and 10,000 testing samples. Character-level tokenization was employed, where each character is treated as an individual token and mapped to a unique integer before being converted into an embedding vector.

8 Model Configuration

Table 1: GPT Model Configurations

Model	d_{model}	Heads	Layers	Context Length
Tiny	64	2	2	128
Small	128	4	4	256
Medium	256	8	6	256

Table 2: Training Data Used for the Small GPT Model

Experiment	Training Data Used
1	10%
2	25%
3	50%
4	100%

9 Scaling Curve

The empirical scaling exponents obtained by fitting the power-law equation are

$$\alpha_N = 0.0178$$

$$\alpha_D = 0.00894$$

The scaling ratio is

$$\gamma = \frac{\alpha_N}{\alpha_D} = 1.9837.$$

10 Model Architecture

The GPT model was implemented entirely in PyTorch. The architecture consists of an input embedding layer, positional encoding, masked multi-head self-attention, feed-forward neural network, pre-layer normalization, residual connections, decoder blocks, and a final linear projection layer for next-token prediction.